

Alpamayo 1.5 RL Training Architecture Guide

A Beginner-Friendly Tutorial

From Model Structure to GRPO Reinforcement Learning

Based on NVIDIA Cosmos-RL Framework

References: [NVLabs/alpamayo](#), [NVLabs/alpamayo1.5](#), [ikutasama/alpamayo1.5](#)

April 2026

> 面向科研新人的入门级讲解——从模型结构到 GRPO 强化学习完整流程

目录

- 1. 先理解：Alpamayo 模型是什么
- 2. 模型的双通道结构
- 3. RL 训练整体架构：Cosmos-RL 框架
- 4. 数据流水线：从 PAI 数据集到训练 Batch
- 5. Rollout 生成：vLLM 推理引擎
- 6. Reward 计算：如何给一条"驾驶轨迹"打分
- 7. GRPO 策略优化：Loss 是怎么更新的
- 8. 权重同步与异步训练
- 9. 核心文件导航
- 10. 总结：一步训练的完整生命周期

1. 先理解：Alpamayo 模型是什么

Alpamayo 是一个端到端自动驾驶模型——输入是多视角相机图像 + 自车历史轨迹，输出是未来的驾驶轨迹。Alpamayo 1.5 相比 Alpamayo 1 增加了：

特性	Alpamayo 1	Alpamayo 1.5
CoC 推理		
导航输入		
VQA 视觉问答		
可变相机数量		

RL 后训练		
------------	--	--

RL
后训练的目的在于：让模型生成的推理文本（chain-of-causation）和轨迹预测在真实驾驶环境中表现更好。直观地说，就是让模型"想得对 + 开得好"。

2. 模型的双通道结构

Alpamayo 模型由两个主要组件构成：

2.1 VLM Backbone (RL 训练的对象)

- 基于 Qwen3-VL，接收多帧多相机图像（最多 4 帧 × 4 相机 = 16 张图）
 - 同时接收历史轨迹 tokens（自行车过去 1.6s 的位置序列）
 - 自回归生成：
 - 先输出推理文本（CoC: 描述场景、风险、决策），截止于 <|cot_end|>
 - 再输出未来轨迹离散 token，截止于 <|traj_future_end|>
- > 关键点：RL 训练只训练 VLM Backbone 的 LM 部分（即文本/轨迹生成的参数），不训练 Action Expert（Diffusion 头）。

2.2 Action Expert

- 将 VLM 最后一层 hidden states 输入 Flow Matching 网络
 - 输出连续的未来轨迹坐标（64 步 × 10Hz = 6.4s 的轨迹）
 - 当前 RL 训练不接触这个模块
-

3. RL 训练整体架构：Cosmos-RL 框架

3.1 为什么用 Cosmos-RL?

Cosmos-RL 是 NVIDIA 开发的大规模异步 RL 训练框架，专为 Physical AI 场景设计。它的核心理念是策略训练和样本生成解耦——训练进程（Policy）和推演进程（Rollout）独立运行在不同的 GPU 上，通过中央控制器协调。

3.2 三大角色

- Controller: 分发 rollout 任务、收集完成的 rollout (prompt + completion + reward)、管理训练 buffer、定时同步 Policy 权重到 Rollout
- Policy Replica: 从 buffer 取 rollout 数据 → 计算 loss → 反向传播 → 更新参数
- Rollout Replica: 用当前模型参数 (vLLM 引擎) 对 prompt 生成 completion → 调用 reward 函数打分 → 反馈给 Controller

3.3 GRPO 算法简介

- GRPO (Group Relative Policy Optimization) 是 PPO 的简化版：
- 对每个 prompt 生成 N 条候选 completion (n_generation, 如 12 条)
 - 在组内计算相对 advantage (好样本有正 advantage, 坏样本有负 advantage)
 - 用 advantage 加权更新策略，鼓励好行为、抑制坏行为
-

4. 数据流水线：从 PAI 数据集到训练 Batch

4.1 PAI 数据集结构

PAI (Physical AI - Autonomous Vehicles) 是 NVIDIA 发布的自动驾驶数据集。每个样本包含：

一个样本 = {

```

image_frames: [4帧, 4相机, 3通道, H, W],    ← 多视角图像
ego_history_xyz: [16步, 3],                  ← 自车历史位置
ego_history_rot: [16步, 3, 3],               ← 自车历史旋转矩阵
ego_future_xyz: [64步, 3],                  ← GT 未来位置 (GT: Ground Truth)
ego_future_rot: [64步, 3, 3],               ← GT 未来旋转
cot: "车辆正从右侧车道切入..."            ← 预标注的推理文本
}

```

4.2 数据预处理流程

原始 PAI 样本

```

tokenized_data = {
    input_ids: [1, L],      ← VLM 的 token 序列
    position_ids: [1, L],
    attention_mask: [1, L],
    pixel_values: [N, C, H, W], ← 图像 tensor
    labels_mask: [1, L],    ← 哪些位置参与 loss 计算
    text: "...",           ← 原始文本 (用于拼接 rollout)
}

```

具体序列结构 (示意) :

```

<|prompt_start|> <|image_pad|> × N <|image_end|>
<|traj_history_start|> [历史轨迹tokens] <|traj_history_end|>
"请预测..." <|cot_start|> [推理文本] <|cot_end|>
<|traj_future_start|> [等待生成] <|traj_future_end|>

```

4.3 关键 Token 的作用

Token	含义
`<	image_pad
`<	traj_history_start/end
`<	cot_start/end
`<	traj_future_start/end

4.4 Node Prefetch 加速

PAI 样本很大 (包含多帧图像), 每个 GPU rank 独立加载会浪费 I/O。Node Prefetch Server 的设计:

效果: 单步训练时间从 44s → 5s。

5. Rollout 生成: vLLM 推理引擎

5.1 Rollout 是做什么的?

Rollout = "让模型在当前参数下, 对给定的驾驶场景 prompt, 生成一条推理+轨迹 completion"。

5.2 生成流程 (代码路径)

```

1. Controller 分配 rollout 任务
   payload = {"idx": "123", "split": "train"}
2. DataPacker.get_rollout_input(payload)
3. vLLM 引擎生成 (n_generation=12 条独立 completion)
   llm.generate(prompts, sampling_params={
       temperature: 0.6,
       top_p: 0.98,
       max_tokens: max_response_length,
       stop_token_ids: [traj_future_end],
       skip_special_tokens: false, ← 保留特殊token在输出文本中!
   })

```

```

}))
4. 过滤空 completion → 打包为 RolloutResult
RolloutResult = {
    prompt: payload,          ← 原始 prompt 索引
    completions: ["推理1... 轨迹1...", "推理2... 轨迹2...", ...],
    cumulative_logprob: [...] ← 每条 completion 的对数概率
}
5. Reward 计算 (见下一章) → 反馈 Controller

```

5.3 vLLM 引擎初始化

```

# rollout.py: ReasoningVlaVllmRollout.init_engine()
LLM(
    model=checkpoint_path,      # Alpamayo 训练级 checkpoint
    hf_overrides=_reasoning_vla_vllm_hf_overrides,
    tensor_parallel_size=tp_size, # 多 GPU tensor 并行
    trust_remote_code=True,
    dtype="auto",
    ...
)

```

vLLM 通过 `hf_overrides` 将 `ReasoningVLAConfig` 转换为 vLLM 可识别的 LLM `text_config`。

6. Reward 计算：如何给一条"驾驶轨迹"打分

Reward 函数是 RL 的核心——它告诉模型什么是"好"的驾驶行为。当前实现包含两个分量：

6.1 轨迹误差 (Trajectory ADE)

```

# traj_reward.py
def calculate_ade(pred_trajectory, gt_trajectory):
    """Average Displacement Error (XY only)"""
    pred_xy = pred_trajectory[..., :2] # 只取 XY 平面坐标
    gt_xy = gt_trajectory[..., :2]
    distances = torch.linalg.norm(pred_xy - gt_xy, dim=-1)
    return float(distances.mean()) # 平均 L2 距离

```

原理：逐时间步计算预测轨迹与 GT 轨迹的欧式距离，取平均。

6.2 舒适度 (Comfort)

```

# comfort_reward.py
COMFORT_METRIC_CONFIG_DICT = {
    "comfort/lon_accel": ["ego_dv_lon", MIN_LON_ACCEL, MAX_LON_ACCEL],
    "comfort/lat_accel": ["ego_dv_lat", -MAX_ABS_LAT_ACCEL, MAX_ABS_LAT_ACCEL],
    "comfort/jerk": ["ego_jerk", -MAX_ABS_MAG_JERK, MAX_ABS_MAG_JERK],
    "comfort/lon_jerk": ["ego_jerk_lon", -MAX_ABS_LON_JERK, MAX_ABS_LON_JERK],
    "comfort/yaw_accel": ["ego_yaw_accel", -MAX_ABS_YAW_ACCEL, MAX_ABS_YAW_ACCEL],
    "comfort/yaw_rate": ["ego_yaw_rate", -MAX_ABS_YAW_RATE, MAX_ABS_YAW_RATE],
}

```

从预测轨迹的 (x, y, yaw) 序列中差分计算：

- 纵向加速度 (m/s^2)
- 横向加速度 (m/s^2)
- Jerk (加速度变化率, m/s^3)
- 横摆角速度 (rad/s)
- 横摆角加速度 (rad/s^2)

每个维度的分数 = 所有时间步都在舒适阈值内的比例 (0~1)。

6.3 组合：Aggregated Reward

```
# aggregated_reward.py
def compute_reward(to_be_evaluated, reference, ...):
    # 1. 从生成的文本中解码出轨迹（切出 traj_future_start/end 之间的 tokens）
    predicted_fut_xyz, predicted_fut_rot = decode_rollout_trajectory(
        to_be_evaluated, reference["ego_history_xyz"], ...
    )

    # 2. 计算轨迹误差
    l2_dist = calculate_ade(predicted_fut_xyz, gt_fut_xyz)

    # 3. 计算舒适度分数
    comfort_score = sum(comfort_dict.values()) / num_metrics - 1.0

    # 4. 门控组合
    if l2_dist < 3.0: # 轨迹误差小于 3 米阈值
        reward = -traj_l2_weight * (l2_dist / 3.0) + comfort_weight * comfort_score
    else:
        reward = -1.0 # 轨迹太差，直接给最低分
```

Reward 的本质：

- 轨迹越准，ADE 越小 → reward 越高
- 驾驶越平顺，舒适度越高 → reward 越高
- 轨迹太差（ADE > 3m）→ 固定 -1 分（这个样本没有学习价值）

6.4 为什么 stop_token_ids 保留特殊 token?

```
# rollout.py: 必须保留!
for sp in (self.val_sampling_params, self.sampling_params):
    sp.skip_special_tokens = False
```

因为 reward 函数需要从生成文本中解析 和 来提取轨迹 tokens。如果 vLLM 去掉了特殊 token，reward 函数无法工作。

6.5 推理文本的 Reward（未来扩展）

当前 reward 只对轨迹部分打分。但 Alpamayo 还生成了推理（CoC）文本。你可以扩展 reward 来同时评价推理质量：

```
reasoning_text = to_be_evaluated.split("<|cot_end|>")[0]
# 用 LLM judge / 规则 / 学习型 reward model 来评价推理
reasoning_score = evaluate_reasoning(reasoning_text)
```

7. GRPO 策略优化：Loss 是怎么更新的

这是整个 RL 训练最核心的部分。我们从 Trainer 代码反向推导数学逻辑。

7.1 一个训练 Step 的完整过程

输入：一批 Rollout (prompt + completion + advantage)

7.2 Model Forward 详解

```
# RLWrapperReasoningVLA.forward()
def forward(self, tokenized_data, ego_history_xyz, ego_history_rot,
            ego_future_xyz, ego_future_rot, labels_mask, **kwargs):

    # Step 1: 将历史轨迹 tokens 注入到 input_ids 中
    input_ids = tokenized_data.pop("input_ids")
    traj_data = {
        "ego_history_xyz": ego_history_xyz,
        "ego_history_rot": ego_history_rot,
        "ego_future_xyz": ego_future_xyz,
        "ego_future_rot": ego_future_rot,
    }
```

```

input_ids = self.fuse_traj_tokens(input_ids, traj_data)
# 输入序列变为: ...<|traj_history_start|>[HIST TOKENS]<|traj_history_end|>...

# Step 2: 准备 labels
labels = input_ids.clone()
if labels_mask is not None:
    labels = torch.where(labels_mask, labels, IGNORE_INDEX)
# IGNORE_INDEX = -100: PyTorch CrossEntropy 会忽略 label=-100 的位置

# Step 3: VLM Forward
outputs = self.vlm(input_ids=input_ids, labels=labels, **tokenized_data)

# Step 4: 分别计算两类 loss
# 4a. 轨迹 loss (traj_future 部分的 NLL)
traj_mask = (labels >= future_start) & (labels < future_start + vocab_size)
future_traj_loss = cross_entropy(logits[traj_mask], labels[traj_mask])

# 4b. 其他所有 token 的 loss (推理文本等)
labels[traj_mask] = IGNORE_INDEX # 把已算过的遮掉
others_loss = cross_entropy(logits, labels)

total_loss = future_traj_weight * future_traj_loss + others_weight * others_loss

```

7.3 GRPO Loss 公式（简化解理解）

GRPO 的损失函数可以理解为（来自 Cosmos-RL 的 compute_loss）：

$$\mathbb{E} \left[\frac{\pi_{\theta}(a|s)}{\pi_{\text{old}}(a|s)} \cdot \beta \cdot D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) \right]$$

主要变量解释：

- π_{θ} ：当前策略（正在训练的模型）
- π_{old} ：旧策略（本 step 开始前冻结的模型）
- π_{ref} ：参考策略（初始 SFT 模型或定期更新的快照）
- $A = \frac{\text{reward} - \text{mean}(\text{reward})}{\text{std}(\text{reward})}$ ：组内归一化的 advantage
- β ：KL 散度的权重系数（kl_beta in TOML config）

为什么要 KL 惩罚？防止模型更新后偏离 SFT 太远，保持基本的语言/推理能力不退化。

7.4 Token-Level Logprob 计算

```

# 对序列中的每个位置计算 log P(token_t | token_{<t})
# logits: [B, L, V] → softmax → log → 取出 target token 位置的值
current_per_token_logprobs, cu_seqlens, _ = compute_logprobs(
    user_mini_batch, logits=raw_logits, is_full_logits=True
)
# 输出: [total_valid_tokens] 形状的 log probability 向量

```

7.5 多次参数更新（μ-iterations）

GRPO 允许一个 batch 多次更新（mu_iterations 参数）。第一次更新时保存 old_per_token_logps，后续迭代都跟这个旧值比较（实现 PPO 的 clipping 效果）。

7.6 Positive NLL Loss（选择性启用）

当 positive_nll_coef > 0 时，对 reward > 0 的好样本额外加一个 NLL（Negative Log-Likelihood）损失，强化好行为的生成概率。

8. 权重同步与异步训练

8.1 同步机制

Policy 训练进程 Rollout 推理进程

sync_weight_interval 的值越小 → rollout 用的权重越新鲜 → 训练更 on-policy，但通信开销更大。实际使用：

- 本地测试：sync_weight_interval = 2
- 集群训练：sync_weight_interval = 5

8.2 异步的优势

- Policy 和 Rollout 完全解耦，不会互相等待
- 当 Policy 在训练时，Rollout 可以并行生成下一批样本
- 类似“流水线”：一边产出数据，一边消费数据

9. 核心文件导航

入口与启动

文件	作用
`finetune/rl/models/reasoning_vla/alpama`	**RL 训练的入口脚本**。注册 ModelSpec (wrapper + ma
`finetune/rl/launcher.py`	共享启动逻辑：从 TOML 读取 checkpoint 路径，调用 `state
`finetune/rl/toml/alpamayo_rvla_rl_local`	TOML 配置文件（控制所有训练超参数）

模型与配置

文件	作用
`finetune/rl/models/reasoning_vla/base_m`	**`RLWrapperReasoningVLA`**：RL 训练用的模型包装器
`finetune/rl/models/reasoning_vla/config`	**`RLWrapperReasoningVLAConfig`**：训练配置类。
`src/alpamayo1_5/models/base_model.py`	基础模型（共享）：`ReasoningVLA`、`TrajectoryFusio
`src/alpamayo1_5/models/alpamayo1_5.py`	Alpamayo 1.5 推理模型（含 diffusion expert）。RL

数据处理

文件	作用
`finetune/rl/models/reasoning_vla/data_p`	**`RVLADataPacker`**：数据打包器。核心职责：① `_samp
`finetune/rl/state.py`	**全局状态管理**：持有 tokenizer、traj_tokenizer、d
`finetune/rl/prefetch/server.py`	**Node Prefetch Server**：每节点一个进程，预加载+预处理
`finetune/rl/hydra_configs/alpamayo1_5_r`	PAI 数据集的 Hydra 配置
`finetune/rl/utils/trajectory_decode.py`	**`decode_rollout_trajectory()`**：从生成的文本

训练与推演

文件	作用
`finetune/rl/models/reasoning_vla/traine`	**`ReasoningVLAGRPOTrainer`**：GRPO 训练器。`
`finetune/rl/models/reasoning_vla/rollou`	**`ReasoningVlaVllmRollout`**：vLLM 推演引擎。
`finetune/rl/models/reasoning_vla/cosmos`	**`RVLACosmos`**：Cosmos 框架适配器，桥接 Cosmos-
`finetune/rl/base_trainer.py`	**`AlpamayoGRPOTrainer`**：公共基类。实现 ref mo
`finetune/rl/models/reasoning_vla/weight`	权重名称映射：policy rollout / HF format

Reward

文件	作用
`finetune/rl/rewards/aggregated_reward.p`	<code>**compute_reward()</code> : Reward 入口。组合 ADE
`finetune/rl/rewards/traj_reward.py`	轨迹 ADE 计算
`finetune/rl/rewards/comfort_reward.py`	舒适度 (加速度/jerk/yaw rate) 计算

10. 总结：一步训练的完整生命周期

下面是一个 RL 训练 step 的完整数据流（从原始数据到参数更新）：

重点记忆

- 1. RL 只训练 VLM Backbone 的 LM 部分，不训练 Diffusion Action Expert
- 2. GRPO 自动计算 advantage：同 prompt 的多条 completion 互相比对，好的往上推、差的往下拉
- 3. Reward = 轨迹准确度 (ADE) + 驾驶舒适度 (加速度/jerk)，轨迹太差 (>3m ADE) 直接 -1 分
- 4. 特殊 token 必须保留在输出中 (skip_special_tokens=False)，否则 reward 函数无法解析轨迹
- 5. Node Prefetch 通过同节点共享内存大幅减少数据加载时间
- 6. Policy 和 Rollout 异步解耦，通过 Central Controller 协调
- 7. KL 散度惩罚 防止模型偏离太远，保持基础能力

> 参考资源

- > - 原始 Alpamayo RL 代码: <https://github.com/NVlabs/alpamayo/tree/main/finetune/rl>
- > - Alpamayo 1.5 模型: <https://github.com/NVlabs/alpamayo1.5>
- > - Cosmos-RL 框架: <https://github.com/NVIDIA/Cosmos-RL>
- > - GRPO 论文: <https://arxiv.org/abs/2402.03300>
- > - 迁移实现: <https://github.com/ikutasama/alpamayo1.5>
- > - Alpamayo 论文: <https://arxiv.org/abs/2511.00088>